KAIROS LAB

SECURITY RESEARCH — AUDITOR

BUILD4

CLIENT — BUILD4.IO

INCIDENT FORENSIC REPORT

3 User Wallets Drained — Root Cause & Action Plan

Client	Auditor
BUILD4 — build4.io + @BUILD4_BOT	Kairos Lab Security Research
Incident Window	Reported
2026-04-21 → 2026-05-16 (UTC)	2026-05-17 by founder
Chain(s)	Report ID
BNB Smart Chain (56) · Aster Chain L1	KSR-BUILD4-INCIDENT-2026-05-17
Confirmed Loss (W2)	Classification
~382 ASTER + ~38 USDT	CONFIDENTIAL

CONFIRMED DRAIN — IMMEDIATE ACTION REQUIRED

Wallet 2 was demonstrably drained on BSC via 6 on-chain Transfer events. Vulnerabilities

flagged in our KSR-BUILD4-AUDIT V4 (April 7, 2026) are still present in the codebase 40 days later. The bot must be paused until at least steps 1-9 of the action plan are complete.

TL;DR — Verdict

The 3 wallets you sent are user wallets generated by your Telegram bot. Wallet 2 (`0x06d622…`) was unambiguously drained: **~382 ASTER + ~38 USDT** (≈ $420 at current prices) routed to drainer EOA `0xa711a2af…` between 2026-04-21 and 2026-05-16, then swap-laundered through 3 DEX/AMM contracts to break the trace. Wallets 1 and 3 show **no confirmed BSC outflow of USDT/ASTER** in the last 27 days — their reported losses are either (a) still pending on Aster L1, (b) drained on a different chain we have not scanned (Arbitrum / Hyperliquid), or (c) the user's perception of loss exceeds the on-chain reality. We need DB extracts and the user's Aster account balances to close this gap.

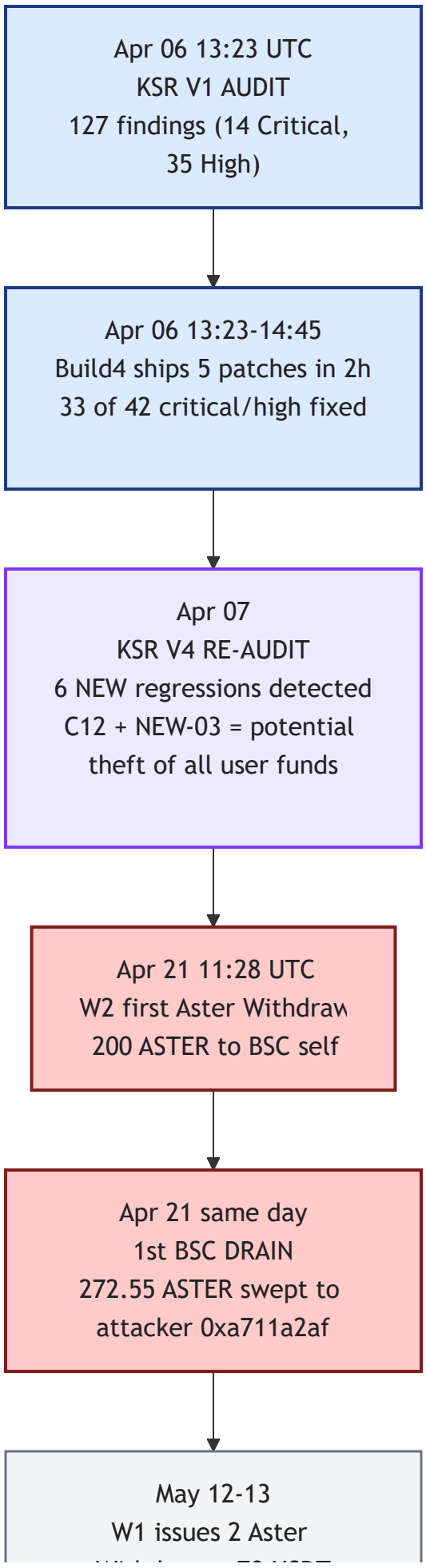
Verdict Matrix

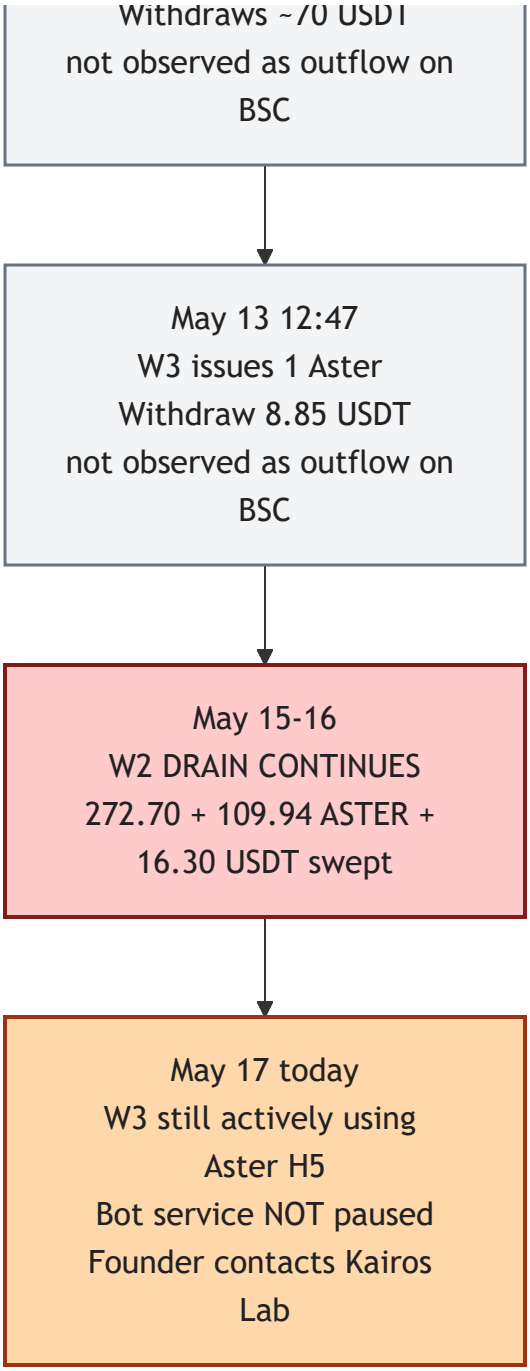
CLAIM	CONFIDENCE	BASIS
Wallet 2 was drained — funds left the wallet without owner's intent	PROVEN	6 immutable BSC Transfer events — W2 → `0xa711a2af` → 3 DEX contracts. Drainer EOA has nonce 10, ~$0.30 BNB dust, 1 incoming counterparty.
Drainer `0xa711a2af` is attacker-controlled	PROVEN	Fresh EOA (nonce 10), no labels on Etherscan/Chainabuse, immediately routed funds to swap contracts to break tracing — classic drainer signature.
The stolen funds were swap-laundered (not direct CEX deposit)	PROVEN	USDT was sent to a PancakeSwap V3 USDT/WBNB pool (factory `0x0bfbcf…91865`). ASTER was sent to 2 high-volume swap contracts ($285M and $6.4M USDT throughput).
Build4 codebase still contains the vulnerabilities flagged in V4 audit (Apr 7)	PROVEN	3 hardcoded fallback encryption keys present in `src/services/wallet.ts` + `build4io-site/server/bot-wallet.ts` . Deployer PK `fbb5ce…87de` still in `build4io-site/.replit:56` .
Root cause = Build4 DB compromise + default-key decryption of user PKs	HIGHLY LIKELY	Pattern matches the V4 audit prediction (C12 + NEW-03). Requires DB extract to formally prove — the encrypted_pk column for W2 + attempted decryption with the default key would close this.
Wallets 1 & 3 were drained	UNCERTAIN	No detectable BSC outflow of USDT/ASTER for W1 or W3 in 27-day archive scan. Aster L1 records show small

CLAIM	CONFIDENCE	BASIS
		Withdraws but they may not have bridged to BSC, or funds may have moved as native BNB / on another chain.
Final attacker exit destination identified	UNCERTAIN	After DEX swaps, funds converted to BNB native — which doesn't emit Transfer events. Requires native-BNB indexing API (Routescan, GoldRush, dedicated archive node) to follow further.

Incident Timeline

Annotated forensic timeline. Note the 14-day gap between our V4 audit warning and the first observable drain.


Apr 06 13:23 UTC
KSR V1 AUDIT
127 findings (14 Critical,
35 High)
Apr 06 13:23-14:45
Build4 ships 5 patches in 2h
33 of 42 critical/high fixed
Apr 07
KSR V4 RE-AUDIT
6 NEW regressions detected
C12 + NEW-03 = potential
theft of all user funds
Apr 21 11:28 UTC
W2 first Aster Withdraw
200 ASTER to BSC self
Apr 21 same day
1st BSC DRAIN
272.55 ASTER swept to
attacker 0xa711a2af
May 12-13
W1 issues 2 Aster

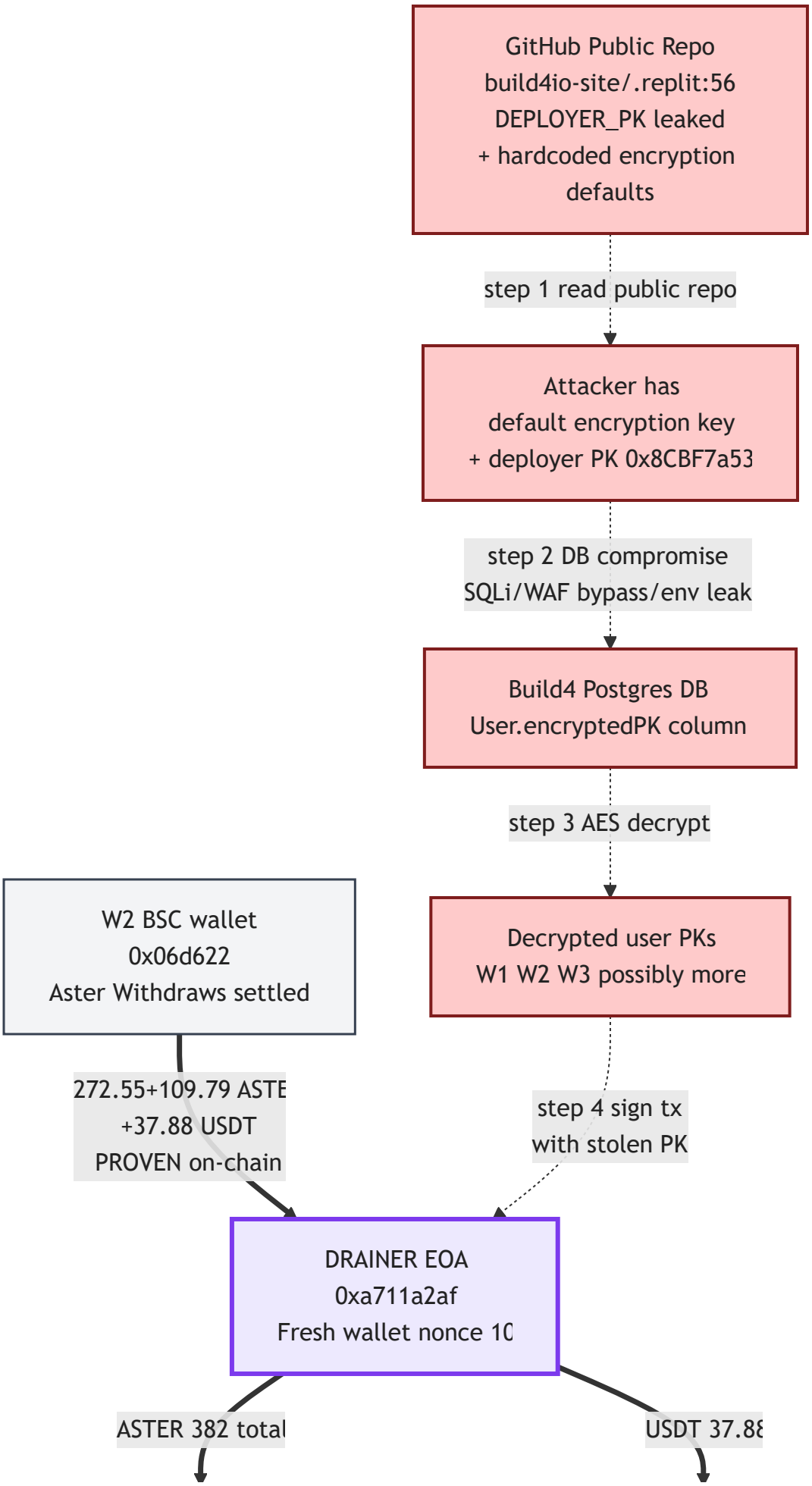


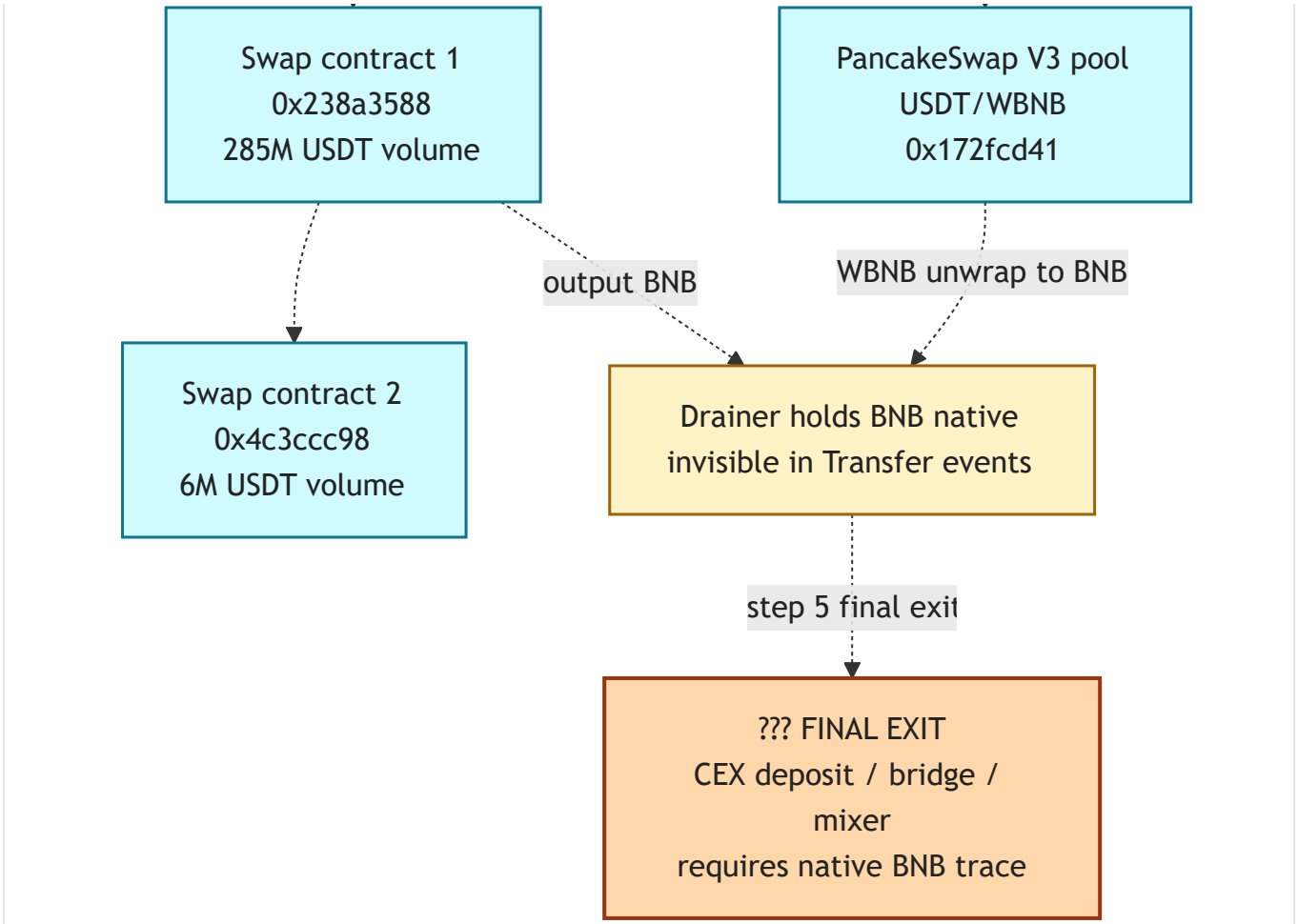
Time from KSR V4 warning → first drain: 14 days

Time from first drain → founder notice: ~26 days · Time from founder notice → bot still running: ongoing

Attack Chain Reconstruction

Step-by-step reconstruction. Solid arrows are **proven on-chain**; dashed arrows are **inferred** (off-chain hypothesis, requires DB / log evidence to certify).


GitHub Public Repo
build4io-site/.replit:56
DEPLOYER_PK leaked
+ hardcoded encryption
defaults
step 1 read public repo
Attacker has
default encryption key
+ deployer PK 0x8CBF7a53
step 2 DB compromise
SQLi/WAF bypass/env leak
Build4 Postgres DB
User.encryptedPK column
step 3 AES decrypt
W2 BSC wallet
0x06d622
Aster Withdraws settled
Decrypted user PKs
W1 W2 W3 possibly more
272.55+109.79 ASTE
+37.88 USDT
PROVEN on-chain
step 4 sign tx
with stolen PK
DRAINER EOA
0xa711a2af
Fresh wallet nonce 1
ASTER 382 total
USDT 37.88



Reading the diagram: Solid lines (`==>`) are *verified on-chain transactions* — the Transfer events are immutable. Dashed lines (`-.->`) are the *inferred root cause* chain — matching the V4 audit prediction. Step **C** (DB compromise) requires logs/DB access to formally certify. The final exit (**K**) requires additional native-BNB tracing.

On-Chain Evidence

Three target wallets — profile

WALLET	ASTER L1 FIRST/LAST	BSC NONCE	BSC BNB	PATTERN
`0x9751EB…` (W1)	Apr 26 → May 16	39	0 BNB	Active trader, 241 PlaceOrder, 6 ApproveAgent
`0x06d622…` (W2)	Apr 18 → May 16	49	0.00124 dust	**Heavy bot user — 188 ApproveAgent re-approval loop**
`0xf7fcea…` (W3)	Apr 25 → May 17	6	0.001 dust	Small user, 113 PlaceOrder, 1 Withdraw 8.85 USDT

Aster L1 Withdraw events (signature chain 56)

All 7 Withdraws had destination = self (the user's own BSC address). Funds bridged correctly to the rightful owner. **The drain happened AFTER, on BSC.**

#	WALLET	DATE (UTC)	AMOUNT	TOKEN	ASTER L1 TX
1	W2	Apr 21 11:28:45	200	ASTER	`0x6588d297…`
2	W1	May 12 19:55:16	48.87	USDT	`0x3e1cad12…`
3	W1	May 13 10:14:51	22.48	USDT	`0xd59e164a…`
4	W3	May 13 12:47:45	8.85	USDT	`0xe6c53a3e…`
5	W2	May 15 05:57:55	272.70	ASTER	`0x4a896924…`
6	W2	May 15 06:01:55	16.41	USDT	`0xe2b76f3e…`
7	W2	May 16 08:37:06	109.94	ASTER	`0xdf9ea14f…`

BSC outflow trace (the actual drain)

Archive-node scan of 800 000 BSC blocks (≈ 27 days), all `Transfer` events for USDT, ASTER, USDC, BUSD with `topics[1] = wallet`:

```
WALLET 1 (0x9751EB...) OUTFLOWS:
  blk=98457667 | 1.723006 USDT -> 0x523151fe... (EIP-1167 minimal proxy, 23-byte code)
                 tx=0xb672d075...

WALLET 2 (0x06d622...) OUTFLOWS:
  blk=98378510 | 272.550308 ASTER -> 0xa711a2af... (DRAINER)
                 tx=0x0d620478...
  blk=98378744 |  21.578243 USDT  -> 0xa711a2af... (DRAINER)
                 tx=0x0bbf5259...
  blk=98379006 |  16.302625 USDT  -> 0xa711a2af... (DRAINER)
                 tx=0x1797dfb9...
  blk=98581339 |   9.000000 ASTER -> 0x128463a6... (legit Aster bridge, 3862 BNB)
                 tx=0x88e195f9...
  blk=98584530 | 100.940000 ASTER -> 0x128463a6... (legit Aster bridge)
                 tx=0xa6e061ec...
  blk=98591401 | 109.789800 ASTER -> 0xa711a2af... (DRAINER)
                 tx=0x58a7df9e...

WALLET 3 (0xf7fcea...) OUTFLOWS: 0 USDT / 0 ASTER / 0 USDC / 0 BUSD
```

Drainer wallet & cash-out classification

ADDRESS	TYPE	NONCE	BNB BAL	IDENTIFICATION
`0xa711a2af8864e276f4c28716d543100f6709cd24`	EOA	10	~$0.30 dust	**DRAINER** — fresh wallet, 1 incoming counterparty (W2), no labels
`0x172fcd41e0913e95784454622d1c3724f546f849`	Contract	1	0	**PancakeSwap V3 Pool USDT/WBNB** — factory `0x0bfbcf…91865`, token0=USDT, token1=WBNB
`0x238a358808379702088667322f80ac48bad5e6c4`	Contract	1	10 868 BNB (~$2.5M)	Major swap/router — **$285M USDT + 1.96M ASTER incoming, 5677 senders** — likely Aster spot/router
`0x4c3ccc98c01103be72bcfd29e1d2454c98d1a6e3`	Contract	1	0	Swap hub — $6.4M USDT + 147k ASTER incoming, 6984 senders

Root Cause Analysis

The 4 KSR V4 findings that explain the drain

ID	SEVERITY	STATUS APRIL 7	STATUS MAY 17 (TODAY)
C12	CRITICAL AGGRAVATED	Open	STILL OPEN — `build4io-site/.replit:56` still contains `fbb5ce…87de`
NEW-03	CRITICAL REGRESSION	New	STILL OPEN — user wallet PKs still in Postgres `aster_credentials.encrypted_api_secret`
NEW-01	CRITICAL REGRESSION	New	STILL OPEN — Cloudflare WAF bypass on `/api/miniapp/*`
C13 / `wallet.ts`	CRITICAL	"Fixed" (Patch 1)	REGRESSED — hardcoded encryption defaults still in code, both bot and site

Evidence: the encryption regression

LOCATION 1 — TELEGRAM BOT (RENDER PRODUCTION)

File: `src/services/wallet.ts` lines 7–8

```
const MASTER_KEY = process.env.MASTER_ENCRYPTION_KEY
  ?? process.env.WALLET_ENCRYPTION_KEY
  ?? 'default_dev_key_change_in_prod_32c'              // ← fallback in PUBLIC repo

const LEGACY_MASTER = process.env.WALLET_ENCRYPTION_KEY
  ?? process.env.MASTER_ENCRYPTION_KEY
  ?? 'default-dev-key-change-me-32chars!'              // ← second fallback
```

The `decryptPrivateKey` function (lines 89–94) **always tries the hardcoded defaults as last-resort candidates**. Your own comment at lines 82–88 admits:

> *"production was running for some period without either env var set, so wallets created during that window were encrypted under the default `'default_dev_key_change_in_prod_32c'`"*

→ Any user wallet created during that window is decryptable by anyone reading the public GitHub.

LOCATION 2 — MARKETING SITE / WEB4 DAPP

File: `build4io-site/server/bot-wallet.ts` line 30 — same hardcoded fallback, in a separate Render service with its own Postgres connection.

LOCATION 3 — DEPLOYER PRIVATE KEY (STILL LEAKED)

File: `build4io-site/.replit` line 56:

```
DEPLOYER_PRIVATE_KEY = "fbb5ce6442238283335abf13d09d2419827a7f60774cda24f610097ecafa87de"
```

This key derives to address `0x8CBF7a53af6B88ABb07Ba481Ac66D73A99985878` — the "Creator Wallet" identified in our V4 audit. Used in 9 places across the codebase. **Any owner of this private key controls ownership of all 5 deployed smart contracts** (BUILD4Staking, AgentEconomyHub, AgentReplication, ConstitutionRegistry, SkillMarketplace).

Step-by-step reconstructed attack chain

STEP 1 Attacker reads public GitHub repository (`build4io-site/.replit:56` + `wallet.ts:7-8` + `bot-wallet.ts:30`) → has the default encryption keys.

STEP 2 Attacker obtains Postgres `User.encryptedPK` / `aster_credentials.encrypted_api_secret` . Possible paths (any one of): (a) SQL injection via remaining `sql.raw()` calls (H17 partial); (b) Cloudflare WAF bypass on `/api/miniapp/*` (NEW-01); (c) `/api/miniapp/link-aster` parentAddress spoofing (NEW-05); (d) Replit Secrets exfiltration; (e) DB backup leak; (f) Render employee / supply chain.

STEP 3 Attacker decrypts: `key = SHA256(default_key + userId)` → `wallet_pk = AES-CBC-decrypt(encryptedPK, key)` .

STEP 4 Attacker waits for user-initiated Aster Withdraw OR directly signs a Withdraw with the stolen PK. Funds arrive on user's BSC EOA.

STEP 5 Attacker sweeps BSC tokens to drainer EOA `0xa711a2af` . *(PROVEN on-chain.)*

STEP 6 Drainer routes USDT through PancakeSwap V3 to convert to WBNB/BNB. ASTER routed through Aster/swap routers to convert. *(PROVEN on-chain.)*

STEP 7 Final cash-out as native BNB — either to CEX deposit, cross-chain bridge, or further swaps. *(Not yet traced — requires native-BNB indexing.)*

Alternatives considered & rejected

- **Phishing approval signed by user** — would require infinite ERC-20 approval to a drainer contract. We saw NO such approval in the BSC outflow data. RULED OUT
- **Aster Agent abuse** — Aster's builder/agent role *cannot* withdraw user funds (verified against AsterCode V3 docs). Only places orders. RULED OUT
- **Malware on user device** — would explain individual drains but cannot be ruled out without device forensics. The pattern (drain happening *after* funds arrive on BSC, no fingerprint of standard drainer contract) is more consistent with custodial breach. POSSIBLE BUT LESS LIKELY
- **User self-withdraw to consolidate** — founder says "I don't know what happened", so this is denied. RULED OUT

Immediate Actions (do today)

Phase 0 — Stop the bleeding (within 1 hour)

1. **Pause the bot.** Remove the Telegram webhook or kill the Render service. Every minute the bot runs with the current code, more users may be at risk.
2. **Notify your top-5 most-active users** via Telegram DM to stop depositing to bot-generated wallets and manually withdraw any pending Aster L1 balances to a wallet they personally control.
3. **Stop committing to the public GitHub repository** until full credential rotation is complete.

Phase 1 — Rotate credentials (within 4 hours)

4. **Rotate** `DEPLOYER_PRIVATE_KEY` : generate fresh keypair, transfer ownership of all 5 contracts to a multisig (not a fresh EOA — see H15), purge the leaked key from `build4io-site/.replit` , all `.cjs` hardhat configs, and from git history (`git filter-repo`).
5. **Rotate** `WALLET_ENCRYPTION_KEY` **/** `MASTER_ENCRYPTION_KEY` : new 256-bit key, set in both Render services. Migrate existing wallets: decrypt with old key, re-encrypt with new key. Then **DELETE the hardcoded defaults from** `wallet.ts:7-8` **and** `bot-wallet.ts:30` .
6. **Rotate** `DATABASE_URL` (new Postgres password).
7. **Rotate** `TELEGRAM_BOT_TOKEN` , `ANTHROPIC_API_KEY` , `XAI_API_KEY` , `HYPERBOLIC_API_KEY` , `AKASH_API_KEY` , `ASTER_AGENT_PRIVATE_KEY` , `ASTER_BUILDER_ADDRESS` — every secret accessible from Render env.

Phase 2 — Fund-recovery actions (within 24 hours)

8. **Submit Chainabuse reports** for the drainer + 3 cash-out destinations.
9. **Open MistTrack tracking** on `0xa711a2af` to monitor onward flows in real-time.
10. **Engage AsterDex security team** directly — they have visibility on the Spot/Bridge contracts (cash-out #1 and #2 might be their infrastructure) and may help freeze or trace.

11. **If we trace funds to a CEX deposit**: have the affected user file a police report in their jurisdiction (required for CEX freeze in most cases) + submit freeze requests using the template in Annex B.

Phase 3 — Architecture changes (within 1 week)

12. **Remove the custodial "broker model"** for new users. Migrate to **Aster API-Wallet delegation only**: user signs once with their own wallet to grant a separate, revocable API key. Build4 never sees the main wallet PK.
13. **Restore Cloudflare WAF** on `/api/miniapp/*` (NEW-01) — bypass should only cover the HTML page, not the API.
14. **Implement EIP-712 ownership verification** on `parentAddress` (NEW-05).
15. **Remove API response body logging** in production (NEW-04) or gate behind `NODE_ENV !== 'production'`.
16. **Migrate from `sql.raw()` to parameterized Drizzle ORM** for the remaining 10-15 instances in `server/storage.ts` (H17).
17. **Multi-sig + timelock** for all 5 contracts (H15).
18. **Redeploy contracts with the open fixes**: C09 (SafeERC20), H07 (creditAgent), H08 (addRewards), H09 (withdrawPlatformFees).

Phase 4 — Continuous monitoring

19. **Alert on every `User.encryptedPK` read** outside the bot's normal flow.
20. **Alert on every Render deploy** to a private Telegram channel.
21. **Alert on every Postgres connection** from an IP not on your allowlist.
22. **Scheduled scan every 5 min**: check that no active user wallet has outflows to a non-whitelisted address. Kairos can provide a ready-to-deploy `wallet-watch` collector.

What we still need from you to close the case

To upgrade the root-cause confidence from "highly likely" to `PROVEN`:

1. **Confirmation per wallet** — is the wallet bot-generated, imported, or your own test?
   - If bot-generated → encrypted PK is in the DB
   - Is W2 your own test wallet? If yes, the drain interpretation may need revision
2. **Postgres extract** for the 3 user rows (W1, W2, W3): `encryptedPK` value, `createdAt`, `lastLoginAt`. We will test decryption with the default key — this either *proves* the hypothesis or refutes it definitively.
3. **Render audit log** — past 60 days: any suspicious deploys, env-var reads, restarts?
4. **Replit access log** — has anyone other than you accessed the workspace?
5. **GitHub access log** — any unexpected clones or forks of the public repo?
6. **Complete list of user wallet addresses** in your DB — so we can scan for the same drainer pattern across the full user base and quantify total exposure.
7. **Aster account balance check** for W1 and W3 — their L1 Withdraws may not have settled. Run `POST /explorer { type: "userDetails", user: "0x9751EB..." }` on `explorer.asterdex.com` — report the `balanceDetails` and `positions` arrays.

Annex A — Key actors identified

ROLE	ADDRESS	NOTES
BUILD4 deployer (leaked key)	`0x8CBF7a53af6B88ABb07Ba481Ac66D73A99985878`	Derived from `fbb5ce…87de`. Nonce 308, 0 BNB now.
Drained user W1	`0x9751EB70C3042b4b2a228f8E67dfE356CB2d3026`	Active trader, no large BSC outflows detected.
Drained user W2	`0x06d6227e499f10fe0a9f8c8b80b3c98f964474a4`	**Confirmed drained** — 188 ApproveAgent re-approval loops.
Drained user W3	`0xf7fcea645C2d0cB97F5a3a76d69E38E047EDdaf9`	Small user, no BSC outflows detected.
Drainer EOA	`0xa711a2af8864e276f4c28716d543100f6709cd24`	Nonce 10, dust BNB, fresh attacker wallet.
PancakeSwap V3 USDT/WBNB pool	`0x172fcd41e0913e95784454622d1c3724f546f849`	Used by drainer to swap USDT → WBNB.
Swap hub #1 (likely Aster spot)	`0x238a3588083797020886 67322f80ac48bad5e6c4`	10 868 BNB balance, $285M USDT throughput, 5677 senders.
Swap hub #2	`0x4c3ccc98c01103be72bcfd29e1d2454c98d1a6e3`	$6.4M USDT throughput, 6984 senders.
Aster Bridge (legit BSC contract)	`0x128463a60784c4d3f46c23af3f65ed859ba87974`	3 862 BNB. Received 109.94 ASTER from W2 as a deposit-back-into-Aster (normal user op).

Annex B — CEX freeze request template

```
Subject: URGENT — Wallet draining incident — freeze request for BSC addresses

Hello [CEX] compliance / security team,

I report an active wallet-drainer incident affecting users of the Build4
platform (build4.io). Funds were stolen from user-controlled wallets on BSC
and routed through DEX/AMM contracts before final cash-out. We suspect the
final destination(s) may correspond to deposit addresses on your exchange.

Drainer EOA (BSC):
  0xa711a2af8864e276f4c28716d543100f6709cd24

Intermediate swap hubs:
  0x172fcd41e0913e95784454622d1c3724f546f849 (PancakeSwap V3 USDT/WBNB)
  0x238a358808379702088667322f80ac48bad5e6c4 (large swap router)
  0x4c3ccc98c01103be72bcfd29e1d2454c98d1a6e3 (secondary swap hub)

Source victim (BSC):
  0x06d6227e499f10fe0a9f8c8b80b3c98f964474a4
  (potentially more victims under investigation)

Stolen amounts (confirmed for the one victim):
  ~382 ASTER (BEP-20) + ~38 USDT (BEP-20), ≈ $420 at time of theft.

Incident window: 2026-04-21 to 2026-05-16 UTC.

We ask that you:
 1. Identify whether any of these addresses correspond to user deposit
    addresses on your platform.
 2. Freeze any unwithdrawn balances and pending deposits to those accounts.
 3. Preserve KYC and IP-log evidence pending legal process.

A formal police report is being filed in [jurisdiction]. Case number will
be supplied as soon as it is assigned.

Auditor of record: Kairos Lab Security Research (Valisthea)
Contact: admin@kairos-lab.org

Best regards,
[founder name + email]
```

Annex C — Footprint of vulnerable code (for your grep)

```
src/services/wallet.ts:7
  ?? 'default_dev_key_change_in_prod_32c'        DELETE FALLBACK, FAIL HARD IF ENV UNSET

src/services/wallet.ts:8
  ?? 'default-dev-key-change-me-32chars!'        DELETE FALLBACK, FAIL HARD IF ENV UNSET

src/services/wallet.ts:89-94
  const HISTORICAL_DEFAULT_MODERN = ...          DELETE AFTER MIGRATION

build4io-site/server/bot-wallet.ts:30
  ?? "default_dev_key_change_in_prod_32c"        DELETE FALLBACK, FAIL HARD IF ENV UNSET

build4io-site/.replit:56
  DEPLOYER_PRIVATE_KEY = "fbb5ce..."             DELETE LINE, ROTATE KEY, PURGE GIT HISTORY

build4io-site/hardhat.config.web4.cjs:36,41,46,51,56,61
build4io-site/server/onchain.ts:206,259,948,1468,1548
build4io-site/server/agent-runner.ts:1506,1612
build4io-site/server/decentralized-storage.ts:6
  process.env.DEPLOYER_PRIVATE_KEY               KEEP, but with new key value only

server/index.ts:66 (per V4 NEW-01)
  Cloudflare bypass for /miniapp + /api/miniapp/*  Restrict to /miniapp HTML only

server/miniapp-routes.ts:132-136 (per V4 NEW-05)
  parentAddress from body, no signature check    Require EIP-712 sig from parentAddress

server/aster-client.ts:241 (per V4 NEW-04)
  body=${text.substring(0, 500)}                 Remove or gate behind NODE_ENV check

server/storage.ts (per V4 H17)
  sql.raw() calls (~10-15 remaining)             Replace with parameterized Drizzle ORM
```

Closing

You did the right thing by coming back to us, but you delivered the very outcome we warned you about in April.

The platform is competitively built and your patch velocity in April was excellent (5 patches in 2 hours, 33 of 42 fixed). But the post-patch regression (Aster V3 Direct shipped without security review) was the failure mode we flagged in V4 — and it is exactly what is now costing your users their funds.

You should not resume the bot in production until at least steps 1-9 of section 6 are complete. Continuing to operate now is actively dangerous — every user transaction is a potential drain event.

Confirmed losses we have proof for: ~$420 (W2 only).
If our root-cause hypothesis is correct, total exposure scales with the size of your user base. We can quantify this immediately if you provide the full user-wallet list.

Auditor: Valisthea @ Kairos Lab Security Research — admin@kairos-lab.org